Prova SB – Teoria (Software Básico / CIC0104) – Resolução comentada

Arquivo original: `ProvaSB_teoria.pdf` (3 páginas). É uma prova **já respondida por um aluno**: tem as marcações V/F/NS, justificativas para os falsos e respostas discursivas (algumas "Não sei"). Abaixo cada marcação foi conferida contra os slides, e as que parecem erradas estão apontadas.

Assuntos cobertos:

- Arquitetura de von Neumann (componentes, ciclo de busca/execução, localidade de referência, gargalo de von Neumann)
- Código absoluto x relocação estática; carregador absoluto (boot)
- Montador (definição, estrutura lógica, síntese/otimização)
- Ligador (definição, etapas, tipos: Linkage Editor, Linking Loader/Binder, ligação dinâmica/DLL)
- Carregador de relocação
- Sistemas operacionais: escalonamento de processos, deadlock/starvation, região crítica, tratamento de E/S (interrupção, programa de canal)
- Gerência de memória real e virtual (partições, swapping, paginação, segmentação, segmentação-paginação, fragmentação)
- Tipos de S.O. (batch, time sharing etc.) e estruturação (questão de projeto)

Como citamos os slides: "7-1" = `7-teoria-1 de 2.pdf`, "7-2" = `7-teoria-2 de 2.pdf`, "8-1" = `8-so-red-1 de 2.pdf`, "8-2" = `8-so-red-2 de 2.pdf`, "9" = `9-boot-so-red.pdf`. "p." é a página do PDF. Em 7-2 e 8-2 a numeração impressa no slide é diferente da página do PDF; quando ajuda, ela aparece entre parênteses (ex.: 7-2 p. 8 (slide 40)).

> **Regras da prova** (tiradas do cabeçalho da outra prova da mesma disciplina e no mesmo formato, `SB_Prova_Sistemas.pdf`, semestre 21/2): preencher V, F ou NS (não sei); **duas erradas anulam uma certa**; em branco conta como errada; **justificar os falsos**; cada item vale 0,32.
>
> **Estratégia:** acerto = +0,32, erro = −0,16, NS = 0. Chutar compensa sempre que a chance de acertar for **maior que 1/3**. Em V/F, se você tem alguma pista, quase sempre vale responder em vez de marcar NS.

> **Atenção:** as provas se repetem com **pequenas alterações** (troca de "páginas" por "segmentos", de "deadlock" por "starvation", troca da justificativa, troca de "é executável" por "não é

> executável"). Ao longo do guia, as variações encontradas em `SB_Prova_Sistemas.pdf` estão anotadas. Leia cada palavra do enunciado.

Parte 1 – Questões objetivas (V / F / NS)

Questão 1

Enunciado: "A arquitetura de von Neumann é caracterizada por CPU (composta por unidade lógica e aritmética, unidade de controle, registradores e contador de programas – PC), memória (RAM onde são armazenados dados e programas), e periféricos de I/O."

Marcação do aluno: **F**. Justificativa dele: "A memória RAM não necessariamente será a memória utilizada."

Resposta: F

Comentário: O slide 7-1 p. 5 define a arquitetura de von Neumann com **quatro** elementos:

- CPU (ULA, UC, registradores, PC);
- Memória (dados e programas);
- Periféricos de E/S (podem ser mapeados na memória);
- **Barramentos (dados, endereço e controle)**.

O enunciado deixa de fora os **barramentos**, então a caracterização está incompleta. Esse é o erro principal. Há um erro secundário: o slide fala só em "memória", não em "RAM". O programa de boot, por exemplo, fica em ROM (7-1 p. 13 e p. 15).

Conferência do gabarito do aluno: a marcação F está certa, mas a justificativa é fraca. A justificativa mais segura é: **"faltam os barramentos (dados, endereço e controle)"**. Se quiser, acrescente que a memória não é necessariamente RAM.

Pegadinha: o professor considera falsa uma definição incompleta. Decore os 4 componentes: **CPU, memória, periféricos de E/S e barramentos**.

Questão 2

Enunciado: "Na arquitetura de von Neumann o processamento é sequencial e segue o ciclo: busca – decodificação – busca de operandos – execução. O carregador ao carregar o programa na memória atribui a PC o valor do **entry point**, passando PC a apontar para uma instrução e esse ciclo é iniciado."

Marcação do aluno: **V**

Resposta: V (provável)

Comentário: O slide 7-1 p. 7 diz: "Processamento é seqüencial. Ciclo busca – decodificação – busca de operandos – execução (fetch–execute cycle) e PC; **PC** ← **Entry point**; PC é incrementado a cada ciclo de busca". O carregador de relocação "transfere o controle para o programa carregado, a partir do entrypoint" (7-2 p. 17 (slide 49)), ou seja, coloca o entry point no PC. Tudo o que o enunciado afirma está correto.

Incerteza: na outra prova (questão 14 de `SB_Prova_Sistemas.pdf`), o mesmo item foi marcado F por outro aluno com a justificativa "não fala que o PC é incrementado". Essa justificativa é fraca, porque o enunciado não afirma nada falso. Mesmo assim, como o professor tende a punir afirmações incompletas (ver Q1), a resposta tem um pouco de risco. A leitura mais defensável é **V**.

Questão 3

Enunciado: "A execução é rápida na arquitetura de von Neumann porque dados e instruções são armazenados em blocos de memória adjacentes devido ao fenômeno de localidade de referência."

Marcação do aluno: **V**

Resposta: V

Comentário: O enunciado é quase uma cópia literal do slide 7-1 p. 7: "Execução rápida porque dados e instruções são armazenados em blocos de memória adjacentes → Localidade de referência". Como dados e instruções ficam próximos e o processamento é sequencial, os acessos se concentram em regiões próximas da memória. Isso é a localidade de referência, que mecanismos como **cache**, buffers e paginação aproveitam.

Observação conceitual: a relação de causa e efeito no enunciado ("armazenados em blocos adjacentes **devido ao** fenômeno de localidade") está um pouco invertida. O mais correto é dizer que o armazenamento adjacente e a execução sequencial **geram** a localidade de referência, e que o

hardware a aproveita. Como o texto segue o slide, o professor quase certamente considera **V**. A outra prova tem o item idêntico (questão 2), também marcado V.

Questão 4

Enunciado: "A redução do gargalo de von Neumann entre CPU e periféricos de I/O pode ser obtida com o uso de memória cache e processadores **pipelining** com múltiplos estágios."

Marcação do aluno: **F**. Justificativa dele: "Além dos citados, necessita dos métodos DMA, bufferização e blocagem."

Resposta: F

Comentário: O slide 7-1 p. 8 divide a redução do gargalo em duas frentes:

- **CPU × memória:** memória cache e processadores com múltiplos estágios (pipelining);
- **Memória × periféricos de E/S:** blocagem e bufferização, DMA e canais de controle de dispositivos de E/S.

O enunciado associa as técnicas de CPU×memória ao gargalo CPU×E/S, e por isso é falso.

Conferência do gabarito do aluno: a marcação está certa, mas a **justificativa está errada**. Ela sugere que cache e pipelining ajudam no gargalo de E/S e que só "faltariam" DMA etc. Isso não é verdade: cache e pipelining **não** atuam no gargalo com E/S. A justificativa correta é: **"Cache e pipelining reduzem o gargalo CPU × memória. O gargalo memória/CPU × E/S é reduzido por blocagem, bufferização, DMA e canais de E/S."**

Pegadinha: troca dos pares de gargalo. Decore a tabela do slide 8.

Questão 5

Enunciado: "Código absoluto foi utilizado no início da computação com sistemas monousuário e não é mais utilizado devido a que os sistemas atuais são multiprogramados ou multiprocessados."

Marcação do aluno: **F**. Justificativa dele: "É utilizado no boot."

Resposta: F

Comentário: O slide 7-1 p. 13 mostra que o código absoluto (posições e endereços fixos e reais) **continua em uso** na "memória ROM em procedimentos de boot. No endereço FFFF:0000 da ROM BIOS tem uma instrução jmp...". O carregador absoluto também é código absoluto (7-1 p. 15), e o slide 9 p. 6 mostra o boot começando pela ROM BIOS.

Conferência do gabarito do aluno: correto. Uma versão mais completa da justificativa: "Código absoluto ainda é usado, por exemplo na ROM BIOS e no carregador absoluto do boot (FFFF:0000)."

Questão 6

Enunciado: "O advento do código relocável permitiu o uso de bibliotecas de código objeto como é o caso do **bytecode** Java que roda em qualquer computador com uma JVM Java instalada."

Marcação do aluno: **V**

Resposta: F (provável; ver incerteza)

Comentário: A primeira parte é verdadeira: "Relocação estática viabiliza a construção de bibliotecas de código objeto" (7-1 p. 14) e "É a existência da relocação que possibilita o uso de bibliotecas de código objeto" (7-1 p. 23). O problema é o **exemplo**:

- bytecode Java **não é código objeto relocável** (código de máquina com lista de endereços relocáveis). É um código intermediário para uma **máquina virtual**;
- o bytecode roda "em qualquer computador" porque a **JVM o interpreta ou compila** naquela máquina, e não por causa da relocação estática.

Portanto, a frase atribui a portabilidade do Java ao código relocável, o que é falso.

Justificativa sugerida: "Bytecode Java não é código objeto relocável. Sua portabilidade vem da máquina virtual (JVM), não da relocação."

Incerteza: o padrão do professor é escrever uma primeira metade verdadeira e um exemplo errado no final. Por isso **F** é a resposta mais provável. Mesmo assim, não há gabarito oficial, e um aluno na outra prova marcou NS nesse item. **Conferência:** o aluno marcou **V**, o que provavelmente está errado.

Questão 7

Enunciado: "Ao se ligar um computador, pressionando o botão de **power on**, um carregador absoluto armazenado em memória volátil é ativado e inicia todo o processo de carga do sistema operacional."

Marcação do aluno: **F**. Justificativa dele: "Um carregador absoluto armazenado em memória **Não** volátil."

Resposta: F

Comentário: O slide 7-1 p. 15 diz que o carregador absoluto é "armazenado em memória **não volátil**, em posição fixa e conhecida (ROM, flash)". A ativação é a "ação do operador ao apertar o botão". Uma memória volátil perderia o carregador ao desligar, e nesse caso não haveria como dar boot. Ver também 7-1 p. 13 ("funciona sem energia...") e 9 p. 6 (ROM BIOS).

Conferência do gabarito do aluno: correto.

Pegadinha: a frase inteira é verdadeira, exceto a palavra **"volátil"**.

Questão 8

Enunciado: "Um montador é um programa que converte um programa em código Assembly em um código objeto relocável (código de máquina), se o programa Assembly estiver correto, e cria um arquivo binário com código de máquina relocável, contendo uma tabela (ou lista) de endereços relocáveis e uma lista de símbolos não resolvidos pelo montador."

Marcação do aluno: **V**

Resposta: V

Comentário: O enunciado é a cópia do slide 7-1 p. 17. Montador: "Programa que converte um programa Assembly em um código objeto relocável (código de máquina) se o programa Assembly estiver correto". Saída: arquivo binário com código de máquina relocável, tabela (ou lista) de endereços relocáveis e lista de referências externas (símbolos não resolvidos pelo montador).

Conferência: correto.

Dica: atenção para uma possível variante falsa que diga "gera código absoluto" ou "gera código executável". O montador gera código **relocável**, e quem resolve as referências externas é o **ligador**.

Questão 9

Enunciado: "Do ponto de vista lógico, em geral um montador é estruturado em um pré-processamento (retira comentários, expande macro e faz a montagem condicional), passo 1 (análises léxica, sintática e semântica) e síntese de código (otimização e geração de código absoluto)."

Marcação do aluno: **NS**

Resposta: F

Comentário: A estrutura está correta: pré-passo (7-1 p. 26), análise léxica, sintática e semântica (7-1 p. 25, 28-30) e síntese com otimização opcional e geração de código (7-1 p. 31). O erro está no final: a síntese **"gera código objeto relocável"** (7-1 p. 31), e não código absoluto. Isso é coerente com a definição do montador (7-1 p. 17, ver Q8).

Justificativa: "O montador gera código objeto **relocável**, não absoluto."

Conferência: o aluno marcou NS. Dava para marcar F com segurança. Na outra prova (questão 9, que usa "passo 2" no lugar de "síntese"), outro aluno marcou F com essa mesma justificativa.

Pegadinha: "absoluto" escondido no fim de uma frase longa e correta.

Questão 10

Enunciado: "As otimizações algorítmicas de código que podem ser realizadas na síntese do código são de tempo de execução ou da alocação de memória."

Marcação do aluno: **V**

Resposta: F (provável; ver incerteza)

Comentário: O slide 7-1 p. 31 diz, sobre a síntese: "Otimiza código (opcional). Otimiza o código intermediário gerado pela análise semântica. **A otimização é do código e não das funções implementadas ao nível do código do programador**." Ou seja, a síntese faz otimização **de código**: elimina instruções redundantes, usa melhor os registradores, reduz tamanho e tempo das instruções. Ela **não faz otimização algorítmica**, que é trocar o algoritmo do programador, e isso cabe a quem escreve o programa. A palavra "algorítmicas" torna a frase falsa.

Justificativa sugerida: "A síntese otimiza o código intermediário, e não os algoritmos (funções) implementados pelo programador."

Incerteza: a frase "otimizações de tempo de execução ou de alocação de memória" é verdadeira se lida como "otimizações de código". Tudo depende de o professor usar "algorítmicas" como pegadinha, e o slide sugere que sim. No arquivo `SB_Prova_Sistemas-EU.pdf`, o mesmo item aparece com anotação "F/NS". **Conferência:** o aluno marcou **V**, o que provavelmente está errado. Se você estiver inseguro, NS é aceitável, mas pelo slide **F** é a aposta melhor.

Questão 11

Enunciado: "Um ligador é um programa que, a partir do conjunto de arquivos contendo códigos objetos relocáveis, resolve as referências externas e gera um arquivo contendo os códigos objetos ligados e com indicação dos endereços relocáveis. Esse arquivo, união dos códigos objetos, é executável."

Marcação do aluno: **V**

Resposta: V

Comentário: O slide 7-2 p. 2 (slide 34) define o ligador como "programa que resolve as referências externas em um programa". Entrada: arquivo objeto com código relocável. Saída: **arquivo binário com código de máquina relocável executável** e endereços relocáveis. Função básica: ligação de módulos e resolução de referências externas.

Conferência: correto.

Pegadinha (variante): na outra prova (questão 21), a frase termina com "**não** é executável", e então é **F**. Lembre: a saída do ligador é executável e continua relocável, porque o carregador de relocação ainda vai relocá-la na carga.

Questão 12

Enunciado: "A função básica do ligador é ligar os módulos e resolver as referências externas, seguindo as seguintes etapas: ler e relocar de forma estática o módulo objeto P; atualizar a tabela global de símbolos com os símbolos internos de P que podem ser referenciados em outros módulos; concatenar P aos módulos anteriores; e resolver as referências externas em P."

Marcação do aluno: **V**

Resposta: V

Comentário: O enunciado é a cópia literal das etapas do slide 7-2 p. 8 (slide 40): ler e relocar o módulo objeto P, atualizar a tabela global de símbolos com os símbolos internos de P, concatenar P aos módulos anteriores e resolver as referências externas em P. Veja também o ligador de dois passos (7-2 p. 10, slide 42): no passo 1 obtém P, calcula k, reloca e inclui os símbolos públicos na tabela global; no passo 2 resolve as referências externas consultando a tabela global.

Conferência: correto.

Dica: uma variante falsa pode trocar a **ordem** das etapas ou dizer "símbolos **externos** de P" no lugar de "internos". A tabela global guarda os símbolos **públicos (internos)** de cada módulo (7-2 p. 15, slide 47).

Questão 13

Enunciado: "Os ligadores podem ser apenas dos tipos **Linkage Editor** (gera um arquivo binário executável que pode ser utilizado novamente), **Linking Loader** ou **Binder** (carrega diretamente o código gerado na memória sem gerar um arquivo binário que possa ser reutilizado), ou carregador de função de biblioteca de ligação dinâmica (tipo DLL)."

Marcação do aluno: **V**

Resposta: V (provável; ver incerteza)

Comentário: O slide 7-2 p. 16 (slide 48), "Tipos de Ligadores", traz:

- **Linkage Editor:** executa a ligação antes da carga, e o programa ligado é gravado em um arquivo (módulo de carga), que pode ser executado várias vezes;
- **Linking Loader ou binder:** depois da ligação o programa é carregado na memória e **não é gerado um arquivo executável**.

O slide 7-2 p. 18 (slide 50) acrescenta o terceiro tipo, **Dynamic linking** (ligação em tempo de execução, DLL). Segundo 7-2 p. 20, é o **carregador** que resolve a relocação das funções da biblioteca dinâmica. As três descrições batem com os slides.

Incerteza: a palavra **"apenas"** é o tipo de palavra que costuma tornar uma frase falsa. Os slides, porém, listam exatamente esses três tipos. A versão da outra prova (questão 13) não tem o "apenas", e lá outro aluno marcou NS, com anotação posterior de V. Pelos slides, **V**. Se o professor quiser pegar no "apenas", o argumento seria que ligadores também são classificados por número de passos (um passo ou dois passos, 7-2 p. 9-10). Mas essa é outra classificação, não outro "tipo".

Questão 14

Enunciado: "O carregador de relocação estática lê o tamanho do programa a carregar; solicita ao sistema operacional (S.O.) a alocação de um bloco contíguo de memória com tamanho suficiente para conter o programa; carrega o programa a partir do endereço dado pelo S.O. relocando os endereços relocáveis; e inicializa o PC com o **entrypoint** do programa."

Marcação do aluno: **V**

Resposta: V

Comentário: É o algoritmo do slide 7-2 p. 17 (slide 49), "Carregador de Relocação" em ambiente monousuário: lê o tamanho do programa e o entrypoint; solicita ao gerente de memória um bloco contíguo; carrega a partir do endereço dado pelo S.O., relocando os endereços relocáveis (endereço efetivo = offset + α, ou seja, soma da constante de relocação k, 7-1 p. 14); transfere o controle a partir do entrypoint.

Conferência: correto.

Dica: o slide pergunta "Qual o problema com esse esquema?". A resposta é que ele exige um **bloco contíguo** do tamanho do programa inteiro. Isso causa fragmentação e impede carregar programas maiores que o maior bloco livre, o que motiva a paginação e o overlay.

Questão 15

Enunciado: "Um escalonamento de processos adequado para um S.O. multiprocessado é o **round-robin** pois permite evitar **starvation**."

Marcação do aluno: **V**

Resposta: V

Comentário: O round-robin (fila circular com time-slice, 8-1 p. 9) dá a cada processo da fila de prontos uma fatia de tempo, um de cada vez. Nenhum processo fica esperando para sempre, então ele **evita starvation**, a "inanição" de um processo que nunca recebe a CPU. É preemptivo e serve bem a ambientes multiprogramados e multiprocessados.

Pegadinha (variante): na outra prova (questão 20), a frase era "... pois permite evitar **deadlock**", e então é **F**. O round-robin trata a distribuição da CPU e não resolve a espera circular por recursos. Ele

evita starvation, não deadlock.

Questão 16

Enunciado: "Sistemas de escalonamento procuram maximizar o **throughput** e minimizar o **turnaround time** e o **response time**. Por isso um bom escalonador deve ser não preemptivo."

Marcação do aluno: **F**. Justificativa dele: "Um bom escalonador deve ser preemptivo."

Resposta: F

Comentário: A primeira frase é cópia do slide 8-1 p. 9 e está correta. A conclusão "por isso deve ser não preemptivo" é falsa. O slide diz apenas que o "escalonamento pode ser preemptivo ou não preemptivo". Além disso, para minimizar o **response time** em sistemas interativos é preciso **preempção** (time-slice, round-robin): um escalonador não preemptivo deixa um processo longo segurando a CPU e piora o tempo de resposta dos demais.

Conferência: a marcação F está certa. A justificativa "deve ser preemptivo" é um pouco forte. Uma justificativa mais segura: "A conclusão não decorre: minimizar o response time exige, em geral, preempção (time-slice). Um escalonador pode ser preemptivo ou não."

Questão 17

Enunciado: "Para maximizar o **throughput** e evitar **deadlock** um processo só pode entrar na fila de prontos se já estiver com todos os recursos que necessitar."

Marcação do aluno: **NS**

Resposta: F

Comentário: Essa é a técnica de **pré-alocação de todos os recursos**, que quebra a condição de "posse e espera" (hold and wait). Ela de fato **evita deadlock**, mas **não maximiza o throughput**, pelo contrário:

- os recursos ficam presos ao processo por muito tempo, mesmo sem uso, e a utilização cai;
- outros processos esperam mais, e menos processos terminam por unidade de tempo;
- pode haver starvation de processos que precisam de muitos recursos.

O assunto aparece em "Gerenciamento de alocação de recursos" (8-1 p. 10: "Mecanismos podem ser implementados com o objetivo de evitar situações de deadlock").

Justificativa: "A pré-alocação evita deadlock, mas reduz o throughput, porque os recursos ficam ociosos presos aos processos."

Conferência: o aluno marcou NS. Dava para marcar F. Na outra prova (questão 16), o mesmo item foi marcado F.

Questão 18

Enunciado: "Região crítica é o trecho do código que objetiva garantir a exclusão mútua de processos."

Marcação do aluno: **V**

Resposta: V (provável)

Comentário: No slide 8-1 p. 10, a região crítica aparece como a facilidade para controlar a alocação de recursos: se o recurso está alocado o processo é bloqueado, e se está livre é alocado. É o mecanismo que garante que apenas um processo por vez use o recurso compartilhado, ou seja, a **exclusão mútua**.

Observação conceitual: na literatura de S.O. (Tanenbaum e outros), região crítica é o trecho de código que **acessa um recurso compartilhado** e que por isso **precisa** ser executado em exclusão mútua. Quem *garante* a exclusão mútua são os mecanismos que a protegem, como semáforos e monitores. A frase é aceitável na terminologia do professor, que usa "região crítica" como o mecanismo. Na outra prova o item idêntico (questão 17) também foi marcado V.

Questão 19

Enunciado: "Em um S.O. do tipo **batch** a melhor gerência de memória real é paginação-segmentação."

Marcação do aluno: **NS**

Resposta: F (provável; depende da visão do professor)

Comentário:

1. **Classificação:** nos slides, paginação, segmentação e segmentação-paginação aparecem como **gerência de memória virtual** (8-2 p. 7, 9 e 10: "Gerência de Memória Virtual: Paginação / Segmentação / Segmentação-paginação"). Os títulos de "Gerência de Memória Real" são overlay, partições, swapping e registrador de base (8-2 p. 3-6). Logo, paginação-segmentação nem é "gerência de memória real" na terminologia do professor.
2. **Adequação:** S.O. batch (processamento em lote, 8-1 p. 4) roda jobs não interativos, conhecidos antecipadamente e executados em sequência. Para isso bastam esquemas mais simples de alocação contígua, como **partições** (fixas, com filas por partição, 8-2 p. 4) ou monoprogramação com relocação estática. Não é preciso o custo de um esquema de três níveis.

Justificativa sugerida: "Paginação-segmentação é gerência de memória **virtual**. Para batch, partições (alocação contígua) são adequadas."

Incerteza: "melhor" é uma palavra subjetiva, e não há gabarito oficial. No arquivo `SB_Prova_Sistemas-EU.pdf` , o item aparece com anotação F e essa mesma justificativa (memória virtual). **Conferência:** o aluno marcou NS. A resposta F é defensável.

Questão 20

Enunciado: "Em um S.O. do tipo time **sharing** como as transações são CPU intensiva, a melhor gerência de memória real é de partições."

Marcação do aluno: **NS**

Resposta: F

Comentário: Há dois erros:

1. Em **time sharing** (8-1 p. 4: "uso de time slice"), as transações são **interativas, curtas e de E/S intensiva** (usuários no terminal). Processamento CPU intensivo é característico de **batch**.
2. A gerência de memória real associada a time sharing nos slides é o **swapping**: "Troca processos via interrupção (time slice)" (8-2 p. 5). Com memória virtual, também se usa paginação. Partições fixas limitam o número de usuários simultâneos e causam fragmentação (8-2 p. 4: "Problema de fragmentação da memória").

Justificativa: "Em time sharing as transações são interativas (E/S intensiva), e a gerência adequada é swapping (troca por time-slice), não partições."

Conferência: o aluno marcou NS. F é seguro.

Dica: guarde os pares: **batch ↔ CPU intensiva ↔ partições/contíguo**; **time sharing ↔ interativo/E/S ↔ swapping (time-slice)/paginação**.

Questão 21

Enunciado: "Gerência de memória baseada em segmentos minimiza a fragmentação interna mas favorece a proteção de código e a fragmentação externa."

(No PDF esse item aparece **duas vezes**, no fim da tabela da p. 2 e repetido no começo da tabela seguinte, provavelmente por causa da quebra de página. É o mesmo item.)

Marcação do aluno: **F**. Justificativa dele: "Aumenta a fragmentação interna."

Resposta: V

Comentário: Na **segmentação**, os blocos têm **tamanho variável**, associado à lógica do programa (8-2 p. 9). Isso tem três consequências:

- **minimiza a fragmentação interna**, porque cada segmento tem exatamente o tamanho necessário e não sobra espaço dentro do bloco;
- **favorece a fragmentação externa**, porque blocos de tamanhos variáveis deixam "buracos" entre eles. Ver 8-1 p. 13: blocos de tamanho variável exigem técnicas de combate à fragmentação (pior, melhor e primeiro ajuste);
- **facilita muito a proteção do código** (8-2 p. 9: "Facilita em muito a proteção do código").

As três afirmações são verdadeiras. O "mas" só contrasta a vantagem com a desvantagem.

Conferência: a marcação **F do aluno está ERRADA**, e a justificativa "aumenta a fragmentação interna" também. Quem tem fragmentação interna é a **paginação** (páginas de tamanho fixo: a última página do processo fica parcialmente vazia). A segmentação tem fragmentação **externa**.

Pegadinha (variante): na outra prova (questão 12), a frase era "Gerência de memória baseada em **páginas** minimiza a fragmentação interna, favorece a proteção de código e a fragmentação externa". Nesse caso é **F**: a paginação **tem** fragmentação interna e **não tem** fragmentação externa, porque qualquer frame livre serve. O professor troca "páginas" por "segmentos" entre as provas:

	Paginação (blocos fixos)	Segmentação (blocos variáveis)
Fragmentação interna	**Sim** (última página)	Mínima
Fragmentação externa	Não	**Sim**
Endereço real	**concatenação** frame ‖ deslocamento	**soma** base + deslocamento
Proteção	Facilita	**Facilita em muito**
Custo operacional	Menor	Maior

Questão 22

Enunciado: "Se um processo que estiver de posse da CPU solicitar um acesso a uma página não disponível na memória será criado um programa de canal para carregar a página em uma **page frame** e o processo ficará bloqueado."

Marcação do aluno: **V**

Resposta: V

Comentário: Essa é a falta de página (page fault). O acesso a uma página ausente vira uma requisição de E/S, que segue o roteiro do slide 7-1 p. 9: a informação não está disponível, o processo é **bloqueado** em uma fila específica, há chamada ao supervisor, que **cria um programa para o canal de E/S** e as controladoras; a operação é feita no periférico, o canal gera uma interrupção e o processo volta para a fila de prontos. O slide 8-1 p. 11 confirma: "As operações a serem executadas em um canal de entrada e saída de dados são definidas em um programa para o canal". A página é carregada em um frame livre ou, se não houver, em um frame liberado por um algoritmo como LRU (8-2 p. 7).

Conferência: correto.

Questão extra (sem número): resposta de uma palavra

Enunciado: "Qual o mecanismo que será utilizado para retirar o processo da fila de bloqueado e o introduzir na fila de prontos quando a requisição de acesso a disco tiver sido atendida? Responda com apenas uma palavra."

Resposta do aluno: **"Escalonador."**

Resposta: Interrupção (provável)

Comentário: Pelo roteiro do slide 7-1 p. 9, quando a operação de E/S termina, o "**canal gera interrupção** → **controlador de interrupções desbloqueia processo** → processo é enfileirado na fila de 'prontos'". O slide 7-1 p. 10 diz o mesmo: "Processo bloqueado acorda ao receber **sinal de interrupção**. Volta para a fila dos processos prontos (ready) à espera da CPU". O mecanismo que move o processo de bloqueado para pronto é a **interrupção**, tratada pelo controlador ou rotina de tratamento de interrupção.

O **escalonador** atua em outra transição: escolhe qual processo da fila de **prontos** vai para **executando** (8-1 p. 9: "O escalonador é responsável por definir o processo a ser executado no momento").

Conferência: a resposta do aluno ("Escalonador") **provavelmente está errada**. **Incerteza:** algum professor poderia aceitar "escalonador" no sentido amplo de gerente de processos, mas, pelos slides desta disciplina, a palavra esperada é **interrupção**.

Resumo da Parte 1 (gabarito proposto x marcação do aluno)

Q	Aluno	Proposto	Situação
1	F	**F**	ok (melhorar justificativa: faltam barramentos)
2	V	**V**	ok (leve incerteza)
3	V	**V**	ok
4	F	**F**	ok (justificativa do aluno está errada)
5	F	**F**	ok
6	V	**F**	**divergente** (incerto)
7	F	**F**	ok

Q	Aluno	Proposto	Situação
8	V	**V**	ok
9	NS	**F**	dava para responder
10	V	**F**	**divergente** (incerto)
11	V	**V**	ok
12	V	**V**	ok
13	V	**V**	ok (incerto pelo "apenas")
14	V	**V**	ok
15	V	**V**	ok
16	F	**F**	ok
17	NS	**F**	dava para responder
18	V	**V**	ok (ressalva conceitual)
19	NS	**F**	dava para responder (incerto)
20	NS	**F**	dava para responder
21	F	**V**	**aluno errou**
22	V	**V**	ok
extra	Escalonador	**Interrupção**	**aluno provavelmente errou**

Parte 2 – Questões discursivas

Discursiva 1 (1,0)

Enunciado: "Caracterize o que é (0,5) e o que causa localidade de referência (0,5)."

Resposta do aluno: "A localidade de referência é a organização da memória de forma contínua, onde o programa é colocado em blocos sequenciais de memória, sendo assim as instruções estarão lado a lado."

Resposta modelo:

O que é (0,5): Localidade de referência é a **tendência de um programa, em um curto intervalo de tempo, acessar repetidamente as mesmas posições de memória (localidade temporal) ou posições vizinhas às acessadas recentemente (localidade espacial)**. Na prática, em cada momento a execução se concentra em uma pequena região da memória.

O que causa (0,5): A causa é a própria **arquitetura de von Neumann** e a forma como programamos para ela (7-1 p. 7):

- **dados e instruções são armazenados em blocos de memória adjacentes**, e o **processamento é sequencial**: o PC é incrementado a cada ciclo de busca, então a próxima instrução está quase sempre no endereço seguinte;
- **laços (loops) e sub-rotinas** reexecutam o mesmo trecho de código e reacessam as mesmas variáveis (localidade temporal);
- **estruturas de dados contíguas**, como vetores, matrizes e registros, são percorridas em sequência (localidade espacial);
- a programação estruturada e modular agrupa código e dados relacionados.

Consequência (vale citar): é a localidade de referência que torna eficazes a **memória cache**, os **buffers**, o disk cache e a **paginação**: carregar um bloco ou página traz junto os dados que provavelmente serão usados em seguida. Por isso a execução é rápida (7-1 p. 7-8, 11-12).

Comentário sobre a resposta do aluno: está incompleta e conceitualmente imprecisa. A resposta do aluno descreve uma **causa** (armazenamento contíguo) como se fosse a **definição**. Localidade de referência é um **comportamento do padrão de acessos** do programa, e não "a organização da memória". Também faltam a localidade temporal (laços) e a ligação com a cache. A resposta provavelmente receberia nota parcial.

Dica: separe sempre "o que é" (padrão de acesso: temporal e espacial) de "o que causa" (von Neumann: armazenamento adjacente, execução sequencial, laços, estruturas contíguas). A questão pontua cada parte separadamente.

Discursiva 2 (1,0)

Enunciado: "Quais são os tipos de carregadores existentes? (0,5) Explique em que instante se dá a relocação dos endereços (0,25) e o que é carregado em cada um dos tipos de carregador que citar (0,25). (1,0)"

Resposta do aluno: "Temos o carregador Absoluto e o carregador de relocação."

Resposta modelo:

Tipos (0,5):

1. **Carregador absoluto** (7-1 p. 15-16): código absoluto, armazenado em **memória não volátil (ROM/flash)** em posição fixa e conhecida, ativado pelo operador ao apertar o botão (no mainframe é o **IPL**, Initial Program Load). Lê registros com o layout (endereço de carga, número de palavras, informação) e os copia para posições fixas; quando n = 0, transfere o controle (`transferto[LA]`).
2. **Carregador de relocação (estática)** (7-2 p. 17, slide 49): carrega na RAM um programa executável que está em disco e o reloca para o endereço de carga fornecido pelo S.O.
3. (Vale citar como complemento) **Linking loader/binder**, que liga e carrega sem gerar arquivo (7-2 p. 16), e o **carregador de bibliotecas de ligação dinâmica (DLL/.so)**, que carrega e reloca a rotina em tempo de execução, quando ela é chamada, usando a tabela de fixup (7-2 p. 18-22).

Instante da relocação (0,25):

- **Absoluto: não há relocação**. Os endereços são fixos e reais, definidos antes, em tempo de programação ou tradução, e o código é carregado exatamente nesses endereços.
- **Relocação estática:** a relocação ocorre **em tempo de carga**, depois que o S.O. informa o endereço base α do bloco contíguo alocado. O carregador soma a constante de relocação k a cada endereço listado na **lista de endereços relocáveis** (endereço efetivo = offset + α, 7-1 p. 14 e 7-2 p. 6).
- **Ligação dinâmica:** em **tempo de execução**, na primeira chamada da rotina ou na carga do programa, conforme o caso.
- (Comparação útil: na **relocação dinâmica**, com registrador de base ou paginação, o endereço é traduzido **durante a execução**, a cada acesso, pelo DAT, 8-2 p. 6.)

O que é carregado (0,25):

- **Absoluto:** o **programa de iniciação (bootstrap)**, ou seja, o início do sistema operacional. Exemplo: ROM BIOS em FFFF:0000 → carrega o MBR em 0000:7C00 → boot loader → NTLDR (9 p. 6, 10-11).
- **Relocação:** um **programa executável de usuário (módulo de carga)** gerado pelo ligador, lido do disco para a RAM.
- **Dinâmico:** as **rotinas da biblioteca dinâmica** (DLL, .so) que o programa usa.

Comentário sobre a resposta do aluno: só listou os tipos (vale no máximo parte dos 0,5) e não respondeu o instante da relocação nem o que cada um carrega. Provavelmente recebeu nota parcial

baixa.

Discursiva 3 (1,0)

Enunciado: "Apresente um exemplo e explique como é realizado o mapeamento de um endereço lógico em um endereço real em um ambiente paginado."

Resposta do aluno: "Não sei."

Resposta modelo (baseada em 8-2 p. 7-8):

Conceito: A memória é dividida em **páginas de tamanho fixo** (alguns KB), e a memória física em molduras (page frames) do mesmo tamanho. O **endereço lógico** gerado pela CPU é dividido em duas partes: **p** (número da página) e **d** (deslocamento dentro da página). O mapeamento é feito em hardware pelo **DAT (Dynamic Address Translator)**, com uma **tabela de mapeamento de páginas** cujo início é apontado por um **registrador de base (RegBase)**:

1. A CPU gera o endereço lógico (p, d).
2. O DAT acessa a tabela de mapeamento de páginas na posição **RegBase + p**.
3. A entrada da tabela fornece **r**, o número do frame real onde a página está, além de um bit de presença e bits de proteção.
4. O endereço real é obtido pela **concatenação** de r com d: **ER = r || d**. O deslocamento não muda, porque a página e o frame têm o mesmo tamanho.
5. Se a página **não estiver na memória** (bit de presença = 0), ocorre uma **falta de página** (interrupção): o processo é bloqueado, um programa de canal carrega a página em um frame (se necessário, um algoritmo como **LRU** escolhe a página a remover) e o processo volta para a fila de prontos (ver Q22).

Por que concatenação e não soma? (pergunta do próprio slide 8-2 p. 8) Como o tamanho da página é uma **potência de 2** e os frames começam em múltiplos desse tamanho, os bits de d ocupam exatamente a parte baixa do endereço. Basta "colar" r na parte alta, sem somador, o que é mais rápido. Na **segmentação**, com blocos de tamanho variável, é preciso **somar** base + deslocamento (8-2 p. 9: "o operador de concatenação não pode ser usado").

Exemplo numérico:

- Endereço lógico de 16 bits, páginas de **4 KB = 2^{12} bytes** → d tem 12 bits e p tem 4 bits (16 páginas lógicas).
- Tabela de mapeamento de páginas do processo:

Página lógica (p)	Frame real (r)	Presente?
0	7	sim
1	3	sim
2	5	sim
3	–	não

- Endereço lógico **0x2ABC** = 0010 1010 1011 1100_2 → **p = 0x2** (4 bits altos), **d = 0xABC** (12 bits baixos).
- A tabela dá página 2 → **frame 5**.
- Endereço real = 5 ‖ ABC = **0x5ABC**. Em decimal: 5 × 4096 + 2748 = 23228.
- Se a CPU acessar 0x3010 (p = 3), a página não está presente: ocorre falta de página e a página 3 é carregada do disco para algum frame livre.

Dica: cite que, para acelerar, o DAT pode usar uma **memória associativa** (TLB) com as entradas mais recentes. O slide 8-1 p. 15 e 8-2 p. 2 trazem "memória endereçável versus memória associativa". Vale citar também que a paginação **facilita a proteção** (bits de proteção por página) e permite **alocação não contígua**: só a página do entrypoint é carregada inicialmente (8-2 p. 7).

Discursiva 4 (2,0)

Enunciado: "Coloque-se na posição de um projetista de sistema operacional. Informe a aplicação característica (**batch**, tempo real, **time sharing**, rede ou distribuído), forma de carregamento de programas na memória (contíguos ou não), tipo (monoprogramado, multiprogramado ou multiprocessado), e estrutura (hierárquica, máquinas virtuais, multiprocessados, distribuídos ou orientados a objetos). Apresente e justifique a política de processos adequada (0,5) indicando como ela evita **deadlock** (0,5) e **starvation** (0,5), e a política de gerência de memória real. (0,5) (2,0)"

Resposta do aluno: "Não Sei."

Comentário geral: a questão é **aberta**. Não existe uma única resposta certa, e a nota depende da **coerência** entre as escolhas e da justificativa de cada item pedido. Responda **todos** os itens, de preferência em tópicos que espelhem o enunciado. Seguem dois modelos. Basta escrever **um**.

Modelo A – S.O. de time sharing (recomendado, é o mais fácil de justificar com os slides)

- **Aplicação característica: time sharing**, com vários usuários interativos e uso de time-slice (8-1 p. 4).
- **Carregamento na memória: não contíguo**, com paginação sob demanda: só a página do entrypoint é carregada no início (8-2 p. 7) e o resto conforme a necessidade. A "tendência atual é alocar por partes, sob demanda", o que facilita a proteção (8-1 p. 14).
- **Tipo: multiprogramado**, com vários processos residentes competindo pela CPU. Pode ser multiprocessado se houver mais de uma CPU.
- **Estrutura: hierárquica (em camadas)**. O nível 0 é o kernel (tratamento de interrupções, escalonamento) e cada nível usa os serviços dos níveis inferiores, o que dá um ótimo nível de abstração (8-2 p. 12).
- **Política de processos (0,5):** escalonamento **preemptivo round-robin com time-slice**, combinado com **prioridades**. A preempção por time-slice garante bom **response time** para os usuários interativos, que é o objetivo de um sistema time sharing (8-1 p. 9: maximizar throughput, minimizar turnaround e response time).
- **Como evita deadlock (0,5):** o round-robin sozinho **não** evita deadlock (ver Q15). É preciso uma política de alocação de recursos:
  - acesso a recursos compartilhados sempre por **região crítica**/exclusão mútua (8-1 p. 10);
  - **prevenção da espera circular**: os recursos são numerados e cada processo só pode pedi-los em **ordem crescente**, o que impede um ciclo de espera;
  - (alternativa: detecção periódica do grafo de espera e recuperação abortando ou fazendo rollback de um processo).
- **Como evita starvation (0,5):** no round-robin todo processo da fila de prontos recebe sua fatia de tempo, porque a fila é circular e ninguém fica para trás indefinidamente. Para as prioridades, usa-se **envelhecimento (aging)**: a prioridade de um processo aumenta com o tempo de espera, e um processo de baixa prioridade acaba sendo escalonado.
- **Gerência de memória real (0,5): swapping** com relocação dinâmica por **registrador de base/DAT**. Os processos são trocados entre memória e disco na interrupção de time-slice, com escalonamento por prioridade (8-2 p. 5-6). Como gerência virtual, usa-se a **paginação** com LRU (8-2 p. 7), que elimina a fragmentação externa.

Modelo B – S.O. batch (alternativa)

- **Aplicação: batch** (processamento em lote, jobs CPU intensivos e não interativos).
- **Carregamento: contíguo**, com relocação estática feita pelo carregador de relocação (7-2 p. 17).
- **Tipo: multiprogramado**.
- **Estrutura:** hierárquica.

- **Política de processos: não preemptiva**, FCFS ou "job mais curto primeiro" (SJF). Maximiza o throughput porque há poucas trocas de contexto, e não há usuário esperando resposta.
- **Deadlock: pré-alocação**: o job declara (no JCL) todos os recursos de que precisa e só entra na fila de prontos quando todos estão disponíveis. Isso quebra a condição de "posse e espera". Note que isso reduz o throughput (ver Q17), um custo aceito em troca da garantia.
- **Starvation:** com FCFS não há starvation, porque a ordem de chegada é respeitada. Com SJF, usa-se **aging** para que jobs longos não esperem para sempre.
- **Gerência de memória real: partições** (fixas, com fila por partição, ou variáveis com primeiro ajuste para permitir o colapso de blocos livres, 8-1 p. 13 e 8-2 p. 4). A fragmentação é combatida com relocação.

Dica de prova: os pontos estão em 4 blocos de 0,5. Ponha um subtítulo para cada um ("Política de processos", "Deadlock", "Starvation", "Memória real") para o corretor achar fácil. Não confunda os conceitos: o **round-robin evita starvation, não deadlock**. O deadlock se resolve com prevenção (pré-alocação ou ordenação de recursos), detecção ou evitação.

Assuntos cobrados nesta prova

Tópico	Questões	Qtd.	Conceitos-chave a dominar
Arquitetura de von Neumann	Obj. 1, 2, 3, 4; Disc. 1	5	4 componentes (CPU, memória, E/S, **barramentos**); ciclo busca-decodificação-busca de operandos-execução; PC ← entry point e incremento; localidade de referência (definição: temporal e espacial; causas); gargalo CPU×memória (cache, pipelining) × gargalo memória×E/S (blocagem, bufferização, DMA, canais)
Código absoluto, relocação e carregadores	Obj. 5, 6, 7, 14; Disc. 2	5	código absoluto ainda usado no boot (ROM, FFFF:0000); relocação estática (offset + α, constante k) viabiliza bibliotecas de código objeto; carregador absoluto em memória **não volátil**, IPL; algoritmo do carregador de relocação; instante da relocação (nenhum / carga / execução)

Tópico	Questões	Qtd.	Conceitos-chave a dominar
Montador	Obj. 8, 9, 10	3	gera código objeto **relocável** (não absoluto) com lista de endereços relocáveis e lista de referências externas; pré-passo, análises léxica, sintática e semântica, síntese; a síntese otimiza **código**, não algoritmos
Ligador	Obj. 11, 12, 13	3	saída **executável** e relocável; 4 etapas (ler/relocar P, tabela global com símbolos internos, concatenar, resolver externas); Linkage Editor × Linking Loader/Binder × ligação dinâmica (DLL, .so)
Escalonamento, deadlock, starvation, região crítica	Obj. 15, 16, 17, 18; Disc. 4	5	round-robin evita **starvation** (não deadlock); objetivos: throughput, turnaround, response time; preemptivo × não preemptivo; pré-alocação evita deadlock mas reduz throughput; região crítica e exclusão mútua; aging
E/S, interrupção, page fault	Obj. 22; extra (1 palavra)	2	roteiro de E/S: bloqueia → programa de canal → interrupção → desbloqueia → fila de prontos; **interrupção** move de bloqueado para pronto; o **escalonador** move de pronto para executando
Gerência de memória (real e virtual)	Obj. 19, 20, 21; Disc. 3, 4	5	memória real (overlay, partições, swapping, registrador de base) × virtual (paginação, segmentação, segmentação-paginação); fragmentação interna (paginação) × externa (segmentação); mapeamento paginado (p, d) → tabela → r ‖ d (concatenação) × segmentação (soma); DAT, LRU, TLB
Tipos e estrutura de S.O.	Obj. 19, 20; Disc. 4	3	batch (CPU intensiva, partições, não preemptivo) × time sharing (interativo, time-slice, swapping/paginação, preemptivo); estruturas hierárquica, máquinas virtuais, multiprocessado, distribuído, orientado a objetos

(Algumas questões contam em mais de um tópico.)

Não caiu nesta prova, mas está nos slides: a sequência de boot do Windows XP (slide 9: POST, MBR em 0000:7C00 com assinatura 55AAh, NTLDR, BOOT.INI e caminhos ARC, NTOSKRNL, HAL). A única ligação desta prova com esse slide é o carregador absoluto/ROM BIOS (Q5, Q7, Disc. 2). Vale revisar caso apareça em outra versão.

Padrões de pegadinha do professor (para todas as provas):

1. Frase copiada do slide com **uma palavra trocada**: volátil/não volátil, relocável/absoluto, executável/não executável, páginas/segmentos, deadlock/starvation, CPU×memória / memória×E/S.
2. Primeira metade verdadeira e **conclusão ou exemplo falso** ("por isso deve ser não preemptivo", "como o bytecode Java").
3. Definição **incompleta** conta como falsa (von Neumann sem barramentos).
4. Na justificativa do F, aponte **exatamente** a palavra ou trecho errado e dê a versão correta.